

6-4 · CDC(Debezium) 실시간 캡처 · 실습 2.5H

커넥터 구축과 변경 이벤트 분석

커넥터를 등록하고 네 가지 op 를 눈으로

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

2.5시간 뒤에는

- ① compose 에 mysql-connect 를 추가하고 binlog 설정을 확인한다
- ② REPLICATION 권한의 debezium 계정과 orders 표를 만든다
- ③ REST API 로 mysql-cdc 커넥터를 등록해 RUNNING 을 확인한다
- ④ $op = r \cdot c \cdot u \cdot d$ 네 가지를 파이썬으로 읽어 정리한다

일부러 호스트명을 localhost 로 틀려 FAILED 의 trace 를 읽어 봅니다

일곱 단계 — compose 추가부터 파이썬 정리까지

6-2 의 compose 가 떠 있는 ~/data-eng/week6 에서 시작합니다 — kafka 서비스는 그대로 둡니다

1. 스택 추가	2. 계정·표	3. 커넥터 등록	4·5. 이벤트 관찰	6·7. 파이썬 정리
mysql · connect binlog 확인	debezium 계정 orders 2행	POST /connectors RUNNING 확인	op = r 스냅샷 c · u · d	cdc_reader.py 앱 수정 없음

mysql·connect 추가와 binlog 확인

compose 추가	mysql:8.0 · debezium/connect:2.7.3.Final
기동	docker compose up -d → docker compose ps
binlog 확인	SHOW VARIABLES LIKE 'log_bin' / 'binlog_format'
Connect 응답	curl -s http://localhost:8083/ head -c 200
플러그인	curl -s .../connector-plugins grep "class"

기대: log_bin=ON · binlog_format=ROW · 플러그인 목록에 io.debezium.connector.mysql.MySqlConnector — Connect 기동은 수십 초

CDC 계정과 orders 표 — Debezium 은 읽기만

1

debezium 계정 — REPLICATION 권한

REPLICATION SLAVE·CLIENT 가 binlog 를 읽는 권한 — 쓰기 권한은 주지 않는다

2

orders 표 — id · customer · amount · status

status 기본값 'CREATED' — user1 12000 · user2 35000 두 행 INSERT

3

확인

SELECT * FROM orders; 로 2행이 조회되면 성공

3306 이 겹치면

**docker stop
de-mysql**

register-mysql.json — 여섯 설정과 REST 등록

설정 키	값	뜻
connector.class	...mysql.MySqlConnector	Debezium MySQL 커넥터
database.hostname	mysql	컨테이너 이름으로 접속
database.user / password	debezium / dbz	2단계에서 만든 계정
topic.prefix	shop	토픽 이름 맨 앞
database.include.list	shop	캡처할 DB
schema.history...servers	kafka:29092	내부 리스너(6-2)

POST --data @register-mysql.json .../connectors 뒤 status 에서 connector·tasks[0] 모두 RUNNING 이면 성공

hostname 을 localhost 로 — FAILED 를 읽는다

정리·회고의 「자주 겪는 오류」 첫 번째를 직접 겪어 봅니다 — 커넥터를 삭제한 뒤 틀린 값으로 다시 등록

삭제	JSON 수정	재등록	FAILED	복구
DELETE 로 mysql-cdc 제거	database.hostname → localhost	POST 뒤 status 확인	trace 첫 줄 접속 오류	mysql 로 고쳐 삭제 → 재등록

네 가지 op 를 터미널 두 개로

실행한 SQL	op	before	after
커넥터 등록 시 이미 있던 2행	r	null	현재 행
INSERT ('user3', 9900)	c	null	넣은 행
UPDATE status='PAID' WHERE id=1	u	status=CREATED	status=PAID
DELETE WHERE id=2	d	지워진 행	null

터미널 1: kafka-console-consumer --topic shop.shop.orders 를 켜 둔 채, 터미널 2 에서 한 줄씩 실행하며 대조

cdc_reader.py — 그리고 「앱을 고치지 않는다」

1 Consumer 로 shop.shop.orders 구독
group.id cdc-reader · auto.offset.reset earliest — 스냅샷부터 다시 읽는다

2 payload 에서 op·before·after 만 찍는다
OP = {r: 스냅샷, c: INSERT, u: UPDATE, d: DELETE} 로 사람이 읽기 좋게

3 7단계 — 평범한 SQL 여러 건
INSERT 2건 + UPDATE amount*1.1 → cdc_reader 에 즉시 나타난다

실행

```
python  
cdc_reader.py
```

이 2.5시간이 끝날 때 확인할 것

- 1 log_bin=ON · binlog_format=ROW · curl 8083 에 버전 JSON
- 2 SELECT * FROM orders 에 2행
- 3 status 의 connector.state.tasks[0].state 가 RUNNING
- 4 토픽 목록에 shop.shop.orders 와 schema-changes.shop
- 5 op=r(before=null) → c · u(CREATED→PAID) · d(after=null)
- 6 cdc_reader 에 네 종류가 한 줄씩 — 7단계 SQL 이 즉시 나타난다

6-4 · 실습 2.5H 마무리

다음은 정리·회고 (0.5H)

Connect REST API 를 한 장으로 정리하고, 6-5 적재를 위해 커넥터와 토픽이 살아 있는지 확인합니다

✓ REST API 여덟 명령 요약

✓ 체크리스트 자가 점검

✓ 자주 겪는 오류 네 가지

✓ 6-5 준비: 커넥터·토픽 상태

가져갈 것

UPDATE 이벤트의 before·after status 값